

DDNS Container

A lightweight Dynamic DNS service designed for home lab environments. The service automatically keeps Cloudflare DNS records synchronized with the home's current public IP address, allowing externally hosted services to remain reachable without manual intervention.

Intent

Problem

Home Internet connections typically receive dynamic public IP addresses from the ISP. When this address changes, externally managed DNS records become invalid, preventing users from reaching services hosted within the home network.

Many ISP-provided routers provide little or no support for customizable Dynamic DNS solutions, resulting in manual DNS updates whenever the public IP changes.

Motivation

The project began with three goals:

- Build a lightweight Dynamic DNS service without coupling it to tunnelling or remote-access functionality.
- Learn how Dynamic DNS works by implementing it from first principles.
- Learn containerization and container image publishing by deploying the solution as a Docker image.

Existing infrastructure within the home lab provided an ideal environment for experimentation and deployment.

Target Audience

Home lab enthusiasts who value simple, reliable, and inexpensive infrastructure.

Vision

Provide a set-and-forget Dynamic DNS solution that automatically maintains external DNS records, improving the reliability of services hosted from a home network while requiring no ongoing user intervention.

Constraints

Assumptions

- Users already possess hardware capable of hosting containers.

Requirements

- Configure once and operate automatically.
- Require no manual intervention during normal operation.

Preferences

- Free to operate aside from domain ownership and existing hardware.
- Minimize deployment complexity.
- Package the solution as a Docker container for portability.

Solution

A containerized service running within the home network periodically retrieves the current public IP address, compares it against a cached value, and updates Cloudflare DNS records whenever a change is detected.

Design

Deployment Context

The service executes on a host within the home network and communicates only with two external dependencies:

- Public IP Lookup Service
- Cloudflare API

This intentionally minimizes external dependencies while keeping deployment simple.

Diagram

- Deployment Context Diagram

Behaviour

The service follows a simple synchronization cycle:

1. Retrieve the current public IP.
2. Compare against the previously cached value.
3. If the address has changed, update Cloudflare DNS.
4. Otherwise perform no action and continue monitoring.

This minimizes unnecessary API requests while ensuring DNS records remain synchronized.

Diagram

- Dynamic DNS Synchronization Sequence Diagram

Implementation

- Github
- Dockerhub

Implementation highlights include:

- Containerized deployment using Docker.
 - Configurable runtime behaviour through environment variables.
 - Automated image publishing to Docker Hub using GitHub Actions.
 - Semantic versioning for container releases.
 - Configurable logging to aid deployment and troubleshooting.
-

Reflection

Outcomes

The project successfully produced a lightweight, self-hosted Dynamic DNS solution capable of automatically maintaining Cloudflare DNS records as the home's public IP changes.

The result removes the need for manual DNS maintenance and improves the reliability of externally accessible services hosted from a home network.

Project Outcomes

- Free to operate apart from domain ownership and existing hardware.
- Zero manual intervention once configured.
- Simplified deployment through containerization.

Learning Outcomes

- Developed an understanding of Dynamic DNS implementations.
- Learned Docker image creation and distribution.
- Published a first Docker image.
- Learned semantic versioning practices.

Emergent Outcomes

- Built an automated CI pipeline for image publishing.
- Established a reusable workflow for container-based projects.